

大语言模型 Post-Training 技术调研报告

Project2_TaskB_Survey

本报告系统调研大语言模型 post-training 的三大核心主题：Fine-tuning（指令微调）、Alignment（对齐）和 Evaluation（系统评估）。从技术原理出发，详细解释每种方法解决什么问题、使用什么数据、优化目标是什么、有什么优点和限制，并辅以数学推导和案例分析。

三份基础笔记：

《Fine-tuning 技术笔记》——微调方法、数学基础、数据格式、超参数调优

《Alignment 技术笔记》——对齐方法的完整数学推导、方法对比、选型指南

《Evaluation Harness 技术深度解析》——评估框架架构、Benchmark 详解、统计检验、局限性分析

目录

大语言模型 Post-Training 技术调研报告 Project2_TaskB_Survey	1
📖 三份基础笔记:	1
1. 引言与背景	6
1.1 为什么需要 Post-Training?	6
1.2 Post-Training 三阶段管线	7
1.3 本报告的组织方式	8
2. 小组分工	9
3. Post-Training 概述	10
3.1 什么是 Post-Training	10
3.2 为什么不能只做 SFT?	10
3.3 评估的必要性	11
4. 第一部分: Fine-tuning (指令微调)	12
4.1 技术概览	12
4.1.1 定义与数学形式	12
4.1.2 SFT 的损失函数详解	12
4.1.3 三种训练范式的数学对比	13
4.2 Full Fine-tuning vs PEFT	13
4.2.1 Full FT 的显存消耗精确分析	13
4.2.2 PEFT 的数学定义	14
4.2.3 四种主流 PEFT 方法的数学本质	15
4.3 案例分析: Alpaca (Self-Instruct 指令微调)	15
4.3.1 动机与背景	15
4.3.2 详细分析	16
4.3.3 三种数据格式详解	16
4.4 案例分析: LoRA (低秩适配微调)	18
4.4.1 理论动机: 本征维度假说	18
4.4.3 低秩分解的 SVD 视角	19
4.4.4 参数量计算示例	19
4.4.5 target_modules 的选择	20
4.4.6 LoRA 改进变体一览	21

4.5 案例分析：QLoRA（量化低秩适配微调）	21
4.5.1 动机	21
4.5.2 详细分析	21
4.5.3 显存对比（完整表格）	22
5. 第二部分：Alignment（对齐）	23
5.1 技术概览	23
5.1.1 Alignment 的定义	23
5.1.2 为什么不只做 SFT?	23
5.1.3 Alignment 的 HHH 框架	24
5.1.4 方法全景图	24
5.1.5 方法速查表	25
5.2 案例分析：InstructGPT / RLHF	25
5.2.1 背景	25
5.2.2 详细分析	26
5.2.3 三阶段数学推导	26
5.2.4 偏好数据收集	28
5.2.5 RLHF 的四个模型	28
5.3 案例分析：DPO（直接偏好优化）	29
5.3.1 核心洞察	29
5.3.2 详细分析	29
5.3.3 核心推导：从 RLHF 到 DPO	30
5.3.4 DPO 损失的直观理解	31
5.3.5 常用偏好数据集	31
5.4 案例分析：DeepSeek-R1 / GRPO	32
5.4.1 新范式：Reasoning-oriented Alignment	32
5.4.2 详细分析	32
5.4.3 GRPO 的直观理解	33
5.4.4 GRPO vs PPO vs DPO 对比	34
5.5 关键取舍与方法组合	34
5.5.1 Alignment 的关键取舍	35
5.5.2 最强模型的训练管线	35
6. 第三部分：Evaluation（系统评估）	36

6.1 技术概览	36
6.1.1 为什么需要系统评估?	36
6.1.2 评估的八大维度	36
6.1.3 Benchmark 的数学定义	36
6.1.4 两种评估模式的数学对比	37
6.2 核心指标详解	38
6.2.1 分类型指标	38
6.2.2 生成型指标	38
6.2.3 代码型指标	38
6.2.4 长度归一化 (acc_norm)	39
6.3 案例分析: MMLU 多项选择评估	39
6.3.1 背景	39
6.3.2 详细分析	40
6.3.3 结果解读	40
6.3.4 分维度分析示例	41
6.4 案例分析: GSM8K 生成式评估	41
6.4.1 背景	41
6.4.2 详细分析	42
6.4.3 答案提取示例	42
6.4.4 异常诊断示例	43
6.5 统计显著性检验	43
6.5.1 McNemar 检验 (配对样本比较)	43
6.5.2 Bootstrap 置信区间	44
6.6 局限性分析	44
6.6.1 Benchmark 污染	44
6.6.2 辛普森悖论	44
7. 综合案例分析: 微调 → 对齐 → 评估 端到端管线	46
7.1 案例场景	46
7.2 阶段一: SFT (LoRA 微调)	46
7.3 阶段二: DPO (偏好对齐)	46
7.4 阶段三: GRPO (推理增强, 可选)	47
7.5 阶段四: 系统评估	47

7.6 迭代决策树	47
8. 方法对比与总结	48
8.1 按目标选型决策树	48
8.2 核心方法一句话总结	48
8.3 发展趋势	49
9. 参考文献	50
核心论文	50
核心开源项目	51
本地参考文档	51

1. 引言与背景

1.1 为什么需要 Post-Training?

大语言模型（LLM）本质上是建立在 **Transformer** 架构上的概率生成模型。预训练阶段通过 **Next Token Prediction** 目标在海量文本上训练：

$$P(x_{n+1}|x_1, x_2, ..., x_n; \theta) = softmax(W \cdot h_n)$$

其中 θ 代表数百亿甚至数千亿的参数， h_n 是最后一层隐藏状态。这种自回归训练范式虽然赋予了模型强大的语言生成能力，但也带来了一个根本性矛盾：

预训练目标与用户期望之间的行为鸿沟（**Behavior Gap**）：

维度	预训练模型的行为	用户期望的行为
交互格式	自由文本续写（"今天天气" → "很好，她走出门..."）	对话式交互（用户问 → 模型答）
指令理解	不理解任务约束	精确遵循指令要求
输出控制	无格式约束	输出 JSON、Markdown、指定风格
知识边界	倾向于编造"看起来合理"的内容	诚实地说"我不知道"
安全性	无差别生成	拒绝有害请求

Post-Training 正是为了弥合这一鸿沟而设计的一系列技术方法的统称，主要包括：

1. Fine-tuning（指令微调 / SFT）——教会模型"回答问题"
2. Alignment（对齐）——让模型输出符合人类偏好
3. Evaluation（系统评估）——衡量模型在各维度的表现，指导迭代方向

🔗 详见《Evaluation Harness 技术深度解析》第 1 节：模型的 loss 下降不一定意味着推理能力提升——这被称为"loss 与能力之间的脱钩"。形式化地：若存在两个模

型 A 和 B，其训练损失 $\mathcal{L}(A) > \mathcal{L}(B)$ ，但在某个下游任务 T 上 A 显著优于 B，则损失函数不足以作为能力的完备代理指标。

1.2 Post-Training 三阶段管线

预训练基座模型 (Base Model)

|



第一阶段: Fine-tuning (SFT)	
目标: 教会模型遵循指令、适应特定领域	
数据: 指令-回复对 (ChatML / Alpaca / ShareGPT)	
方法: Full FT / LoRA / QLoRA	
数学: 最小化交叉熵损失 (仅计算回复部分)	
产出: 指令模型 (Instruct Model)	

|



第二阶段: Alignment (对齐)	
目标: 让输出符合人类偏好 (有用、无害、诚实)	
数据: 偏好对 (chosen vs rejected)	
方法: RLHF (PPO) / DPO / GRPO	
数学: 最大化奖励 + KL 约束 / 偏好分类损失 / 组内比较	
产出: 对齐模型 (Aligned Model)	

|



第三阶段: Evaluation (系统评估)	
目标: 系统衡量模型在各维度的能力边界	
数据: 标准化 Benchmark (MMLU/GSM8K/等)	
方法: lm-evaluation-harness / LLM-as-Judge	
数学: Log-likelihood / 生成式匹配 / 统计检验	

产出：评估报告 → 微调/对齐的迭代方向

💡 核心认知：这三个阶段不是一次性的流水线，而是一个迭代循环——评估结果反馈到微调和对齐的数据和策略选择中。最强的模型（如 ChatGPT、Claude、DeepSeek-R1）都经过多轮这样的迭代。

1.3 本报告的组织方式

每个方法/案例都按以下框架展开：

① 解决什么问题 — 动机与定位
② 输入数据 — 数据格式、来源、规模
③ 输出结果 — 产出什么
④ 数学原理 — 优化目标 + 关键公式
⑤ 基本流程 — 关键步骤
⑥ 优点 — 核心优势
⑦ 局限性 — 主要缺点和挑战
⑧ Post-training 位置 — 在管线中的定位

2. 小组分工

本调研报告由以下小组成员协作完成：

成员	角色	负责内容
胡亚鑫	组长 / Fine-tuning 负责人	① Fine-tuning 技术调研与报告撰写；② SFT / LoRA / QLoRA 案例分析与数学推导；③ Alpaca 案例研究；④ 数据格式与超参数字节
赵子仪	Alignment 负责人	① Alignment 技术调研与报告撰写；② RLHF / DPO / GRPO 案例分析与公式推导；③ 方法对比与选型指南；④ 偏好数据集调研
赵子仪	Evaluation 负责人	① Evaluation 技术调研与报告撰写；② lm-eval 工具使用与命令示例；③ Benchmark 体系整理与对比；④ 统计检验与异常诊断
胡亚鑫	综合与整合	① 报告统稿与格式校对；② 参考文献整理；③ 端到端管线案例分析；④ GitHub 仓库管理与提交

协作流程：



3. Post-Training 概述

3.1 什么是 Post-Training

Post-Training（训练后优化）是指在预训练完成后，对模型进行的一系列针对性的优化步骤。它在概念上区别于预训练（Pre-training）的两个关键点：

对比维度	Pre-training	Post-Training
数据	TB 级无标注文本（CommonCrawl, Wikipedia, 书籍）	标注数据（指令-回复对 / 偏好对 / Benchmark）
目标	学习通用语言分布 $P(text)$	校准条件分布 $P(response instruction)$
参数更新	从随机初始化开始	从预训练权重 θ_0 开始
所需资源	数千 GPU 天，数百万美元	数 GPU 天到数 GPU 周，100 ~ 10,000
核心挑战	训练稳定性、数据质量、规模扩展	数据效率、灾难性遗忘、偏好对齐

数学统一视角：所有 Post-Training 方法都可以看作是在调整条件分布：

$$P_{\text{post-train}}(response|instruction; \theta)$$

使得它从预训练的"自由文本续写分布" $P_{PT}(text|prefix)$ 逐渐逼近"理想的助手分布" $P_{ideal}(response|instruction)$ 。

3.2 为什么不能只做 SFT?

SFT 虽然教会了模型"回答问题"，但它有一个致命弱点：

$$\mathcal{L}_{SFT} = - \mathbb{E}_{(x,y^*) \sim \mathcal{D}_{SFT}} [\log \pi_{SFT}(y^*|x)]$$

这个目标函数对所有"非参考答案"的惩罚是均等的——无论一个错误的回答是"略微不准确"还是"极其危险", 在 SFT 的损失函数中都只是"不是 y^* ":

损失函数视角下的"SFT 盲区":

```
SFT loss = -log P("这是一个好问题" | prompt)
          = -log P("这是一个坏问题" | prompt)
          = -log P("我来教你如何制作炸弹" | prompt)
```

三者 loss 一样大! SFT 无法区分"不好的回答"和"危险的回答"!

这就是为什么 SFT 之后必须做 Alignment——Alignment 教会模型"怎么做好"和"什么不能做"。

🔗 详见《Alignment 技术笔记》第 1.3 节: SFT 的局限包括不懂拒绝、不懂"不知道"、不懂偏好、不懂分寸、分布外脆弱。

3.3 评估的必要性

没有任何单一指标可以覆盖所有评估维度。核心矛盾:

不存在一个静态指标能预言模型在未知任务上的表现。

形式化地, 定义一个评估维度集合 $\mathcal{T} = \{T_1, T_2, \dots, T_k\}$, 我们需要:

1. 对于每个维度 T_i , 定义适当的 *metric* M_i
2. 选择合适的评估方式 (Log-likelihood 或生成式)
3. 确保评估结果可复现 (固定 seed、prompt 模板、解码策略)

🔗 详见《Evaluation Harness 技术深度解析》第 1.2 节: 评估的八个核心维度 (知识、数学、代码、文本理解、安全、指令遵循、Agent、多语言)。

4. 第一部分：Fine-tuning（指令微调）

4.1 技术概览

4.1.1 定义与数学形式

Fine-tuning 是指在预训练模型参数 θ_0 的基础上，使用特定任务的标注数据 $\mathcal{D}_{FT} = \{(x_i, y_i)\}_{i=1}^N$ 继续训练：

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{FT}(\theta; \mathcal{D}_{FT}), \quad \text{初始化 } \theta = \theta_0$$

核心思想：迁移学习——预训练阶段学到的通用语言知识（语法、语义、推理能力）已经编码在 θ_0 中，微调只需在 θ_0 附近做局部优化，找到适应下游任务的方向。

4.1.2 SFT 的损失函数详解

对于生成式任务（LLM 微调），训练目标与预训练相同——Next Token Prediction，但数据分布从通用文本变为指令-回复对：

$$\mathcal{L}_{SFT} = -\frac{1}{N} \sum_{i=1}^N \sum_{t=1}^{T_i} \log P_{\theta}(y_{i,t} | x_i, y_{i,<t})$$

关键细节——Loss Masking:

在实际实现中，仅计算 assistant 回复部分的损失，user 指令部分被 mask 掉：

[Prompt]: "User: 什么是机器学习? \nAssistant: 机器学习是人工智能的一个分支..."	
.....	
mask (不计算 loss)	计算 loss (仅这部分)

数学上，loss mask 的实现为：

$$\mathcal{L}_{SFT} = -\frac{1}{\sum_i M_i} \sum_{i=1}^N M_i \cdot \log P_{\theta}(y_i | x_{<i})$$

其中 $M_i \in \{0,1\}$ 是 mask 向量， $M_i = 1$ 当且仅当位置 i 属于 assistant 回复。

4.1.3 三种训练范式的数学对比

范式	损失函数	特点
自回归 LM (GPT)	$\mathcal{L}_{AR} = - \sum_{t=1}^T \log P_{\theta}(x_t x_{1:t-1})$	单向上下文， 天然生成
掩码 LM (BERT)	$\mathcal{L}_{MLM} = - \sum_{i \in \mathcal{M}} \log P_{\theta}(x_i x_{\setminus \mathcal{M}})$	双向上下文， 天然理解
前缀 LM (T5)	$\mathcal{L}_{Prefix} = - \sum_{t= p +1}^{ p + t } \log P_{\theta}(x_t p, x_{<t})$	双向编码 + 自 回归生成

🔗 详见《Fine-tuning 技术笔记》第 1.2 节和第 3.3 节：预训练目标函数对比及 Fine-tuning 的数学定义。

4.2 Full Fine-tuning vs PEFT

4.2.1 Full FT 的显存消耗精确分析

Full FT 的显存开销由四部分组成，这是理解为什么需要 PEFT 的关键：

$$TotalMemory \approx \underbrace{P}_{\text{模型参数}} + \underbrace{P}_{\text{梯度}} + \underbrace{2P}_{\text{Adam 优化器状态}} + \underbrace{\text{激活值}}_{\mathcal{O}(B \cdot L \cdot d \cdot n_{layers})}$$

Adam 优化器为什么需要额外 2P 参数？

Adam 的更新规则需要维护两个额外的状态变量：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{一阶矩估计, FP32})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{二阶矩估计, FP32})$$

完整显存分解（7B 模型）：

组件	精度	大小	说明
模型参数 θ	FP16/BF16	~14 GB	前向和反向传播
梯度 $\nabla_{\theta}\mathcal{L}$	FP16/BF16	~14 GB	反向传播产生
Adam m_t (FP32 主副本)	FP32	~28 GB	一阶矩
Adam v_t (FP32 主副本)	FP32	~28 GB	二阶矩
激活值	FP16	~28-56 GB	取决于 batch size
合计 (不含激活值)		~84 GB	
合计 (含激活值)		~112-140 GB	

这就是 $7 \times 16 \text{ bytes} \approx 112 \text{ GB}$ 经验公式的来源。

4.2.2 PEFT 的数学定义

PEFT (Parameter-Efficient Fine-tuning) 的核心思想是冻结绝大多数预训练参数，仅优化极少量的额外参数 ϕ :

$$\theta_{FT} = \theta_0 \cup \phi^*, \quad \phi^* = \arg \min_{\phi} \mathcal{L}(\theta_0, \phi; \mathcal{D}_{FT})$$

其中 θ_0 冻结不变， ϕ 为可训练的新参数， $|\phi| \ll |\theta_0|$ 。

从优化视角看，PEFT 将搜索限制在低维子流形上：

1. Full FT: 在 $\mathbb{R}^{|\theta|}$ 中搜索 (高维，灵活但易过拟合)
2. PEFT: 在 $\{\theta_0 + \Delta\theta(\phi)\}$ 上搜索的结构约束 (低维，正则化)

这解释了为什么 PEFT 在小数据集上不容易过拟合——低维搜索空间天然具有正则化效应。

4.2.3 四种主流 PEFT 方法的数学本质

方法	参数化方式	可训练参数占比	推理开销
Adapter	$\mathbf{h}_{out} = \mathbf{h}_{in} + f_{act}(\mathbf{h}_{in}W_{down})W_{up}$	~2%-5%	有（串行瓶颈层）
Prefix Tuning	$\mathbf{K}_{aug} = [\mathbf{P}_K; \mathbf{K}], \mathbf{V}_{aug} = [\mathbf{P}_V; \mathbf{V}]$	~0.1%-1%	有（KV cache 增长）
Prompt Tuning	$\mathbf{E}_{aug} = [\mathbf{P}; \mathbf{E}(x)]$	~0.001%-0.01%	无
LoRA	$h = W_0x + \frac{\alpha}{r}BAx$	~0.1%-1%	无（可合并）

🔗 详见《Fine-tuning 技术笔记》第 4 节：完整对比表格和每种方法的详细分析。

4.3 案例分析：Alpaca（Self-Instruct 指令微调）

4.3.1 动机与背景

Stanford Alpaca（2023 年 3 月）是首个以极低成本（~\$100）复现 GPT-3.5 级别指令跟随能力的开源模型。它的核心创新是用 模型生成训练数据（Self-Instruct），解决了高质量指令数据的瓶颈问题。

4.3.2 详细分析

维度	说明
解决什么问题	高质量指令数据稀缺且昂贵（人工标注一条指令-回复对约 \$1-5）。Alpaca 证明：用强大模型自动生成指令数据 + 在开源基座模型上微调，可以以极低成本获得接近 GPT-3.5 的指令跟随能力
输入数据	52K 条 instruction-input-output 三元组。通过 Self-Instruct 管线生成：175 条种子指令 → 调用 text-davinci-003 扩增 → ROUGE-L 去重 → 质量过滤
输出结果	Alpaca 7B 指令模型。全量微调 LLaMA 7B，3 epochs，8×A100 80GB，~3 小时完成
数学原理	Self-Instruct 是一种数据增强——在语义空间做扩展而非特征空间变换：教师模型 M_T （GPT-3.5）基于种子分布 P_{seed} 生成 $D_{synth} = \{(x, M_T(x)) x \sim P_{expansion}(P_{seed})\}$ 。学生模型 M_S （LLaMA 7B）在合成数据上做监督学习，目标是最小化 $\mathcal{L}_{SFT} = -\sum \log P_{M_S}(y x)$
基本流程	① 人工编写 175 条种子指令（覆盖写作、编程、问答等）；② 将种子任务作为 few-shot 示例，调用 text-davinci-003（GPT-3.5）生成新指令；③ 判断是否为分类任务——是则先生成类别，再生成 input；④ 调用 GPT-3.5 为每条指令生成 output；⑤ 过滤（去重、质量检查），得到 52K 条数据；⑥ 用标准 SFT 损失微调 LLaMA 7B
优点	① 成本极低（\$100 + 3 小时），大幅降低指令微调门槛；② 数据质量压倒数量（过滤后的 52K 效果远超更大但更嘈杂的数据集）；③ 验证了“教师模型蒸馏”的可行性——GPT-3.5 的 RLHF 对齐风格通过蒸馏传递给了 Alpaca
局限性	① LLaMA 非商用协议限制实际应用；② 蒸馏争议（OpenAI 服务条款禁止用 API 输出训练竞争模型）；③ 未经 RLHF，幻觉率显著高于 ChatGPT；④ 安全未对齐，易被诱导生成有害内容；⑤ 种子指令覆盖面有限（偏重英语、编程、常识）
Post-training 位置	指令微调（SFT）的代表性案例，展示了用 Self-Instruct 解决数据瓶颈——属于 Fine-tuning 阶段的核心实践

4.3.3 三种数据格式详解

微调数据的格式直接影响模型的行为。以下是三种主流格式：

Alpaca 格式（单轮指令）：


```
{
  "instruction": "将以下句子翻译成英文。",
  "input": "今天天气真好。",
  "output": "The weather is really nice today."
}
```

ShareGPT 格式（多轮对话）：

```
{
  "conversations": [
    {"from": "human", "value": "你好，请帮我写一个快速排序。"},
    {"from": "gpt", "value": "当然，以下是 Python 实现的快速排序：\n```python\ndef\nquicksort(arr):\n    ...```"},
    {"from": "human", "value": "能加上注释吗？"},
    {"from": "gpt", "value": "好的，这是带注释的版本：\n```python\ndef quicksort(arr):\n# 递归终止条件\n    ...```"}
  ]
}
```

ChatML 格式（业界标准，推荐）：

```
{
  "messages": [
    {"role": "system", "content": "你是一个有帮助的 AI 助手。"},
    {"role": "user", "content": "什么是快速排序？"},
    {"role": "assistant", "content": "快速排序是一种分治排序算法，其核心思想是..."}
  ]
}
```

 详见《Fine-tuning 技术笔记》第 8 节：三种数据格式的详细对比和选择建议。

4.4 案例分析：LoRA（低秩适配微调）

4.4.1 理论动机：本征维度假说

LoRA 的设计深受 本征维度（Intrinsic Dimension）研究启发。Aghajanyan et al. (2020) 发现：

即使模型有数十亿参数，其参数更新的有效自由度（即本征维度）远小于参数总量。对于大多数下游任务，在一个仅几百维的随机子空间中优化就足以达到全参数微调 90% 以上的效果。

这暗示一个关键性质： $\text{rank}(\Delta W) \ll \min(d, k)$

其中 ΔW 是微调带来的权重变化量。LoRA 正是利用这一洞察，显式地将 ΔW 建模为两个低秩矩阵的乘积。

4.4.2 详细分析

维度	说明
解决什么问题	Full FT 的显存和存储成本过高（7B 模型需 ~112 GB，每任务需保存完整 ~14GB 模型）。LoRA 将可训练参数量降低到 0.1-1%，同时保持效果接近 Full FT
输入数据	与 Full FT 相同的指令-回复对。LoRA 对数据量不敏感——500-10K 条均可，且由于低秩约束提供天然正则化，小数据场景比 Full FT 更鲁棒
输出结果	一套低秩适配器权重 A, B（~16 MB for 7B model）。可合并回原始权重实现零额外开销推理，也可热插拔用于多任务切换
数学原理	核心公式： $h = W_0 x + \frac{\alpha}{r} B A x$ 。其中 $W_0 \in \mathbb{R}^{d \times k}$ 冻结， $B \in \mathbb{R}^{d \times r}$ 和 $A \in \mathbb{R}^{r \times k}$ 可训练， $r \ll \min(d, k)$ 。初始化为 $A \sim \mathcal{N}(0, \sigma^2)$ （Kaiming）， $B = 0$ （保证训练开始时 $\Delta W = 0$ ）。推理时合并： $W_{merged} = W_0 + \frac{\alpha}{r} B A$
优化目标	与 SFT 相同——交叉熵损失 $\mathcal{L} = -\sum \log P(y_t x, y_{1:t}; W_0, A, B)$ 。但仅优化 A, B， W_0 保持冻结
基本流程	① 冻结预训练权重 W_0 ；② 在目标层（通常为 Attention 投影矩阵）插入低秩矩阵 B, A；③ 仅优化 A, B；④ 保存时仅存储 A, B；⑤ 推理时合并或保持双分支计算

维度	说明
优点	① 显存降低 7×（7B: 16GB vs Full FT ~112GB）；② 存储极轻量（16MB vs ~14GB）；③ 推理零额外开销（可合并）；④ 可热插拔多任务切换；⑤ 低秩约束提供隐式正则化；⑥ 多种改进变体（DoRA, LoRA+, PiSSA）进一步提升效果
局限性	① 复杂任务效果上限略低于 Full FT（通常 < 3%）；② 需选择 target_modules 和秩 r，有调参成本；③ 训练时仍需加载完整基座模型
Post-training 位置	参数高效微调（PEFT）的核心方法，广泛应用于 Fine-tuning 阶段的指令微调 and 领域适应任务

4.4.3 低秩分解的 SVD 视角

任意矩阵 $M \in \mathbb{R}^{d \times k}$ 的奇异值分解：

$$M = U \Sigma V^T = \sum_{i=1}^{\min(d,k)} \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

其中 $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{\min(d,k)} \geq 0$ 为奇异值。如果大多数奇异值接近于 0，则可以用前 r 个最大的奇异值近似：

$$M \approx \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

近似误差由 Eckart-Young-Mirsky 定理 给出：

$$\min_{\text{rank}(M_r) \leq r} \|M - M_r\|_F = \sqrt{\sum_{i=r+1}^{\min(d,k)} \sigma_i^2}$$

LoRA 的洞见：微调中的权重更新 ΔW 恰好具有这种低秩性质，因此可以直接用 BA 来参数化。

4.4.4 参数量计算示例

对于 Transformer 中的一个线性层 $W \in \mathbb{R}^{d \times k}$ （如 $d = k = 4096$ ）：

原始参数量： $d \times k = 4096^2 \approx 16.8M$

LoRA 参数量 (r=8) : $d \times r + r \times k = 4096 \times 8 + 8 \times 4096 = 65,536 \approx 0.066M$

占比: $65,536/16,777,216 \approx 0.39\%$

对于 LLaMA-7B (~6.7B 参数, 32 层, 每层 4 个 Attention 投影矩阵):

全量参数: ~6.7B

LoRA 参数 (r=8, 仅 q,v) : ~4.2M (约 0.06%)

4.4.5 target_modules 的选择

各投影矩阵在 Attention 中的作用:

矩阵	公式	作用	LoRA 推荐
W _q (query)	$Q = W_q X$	决定"我要查什么"	★ ★ ★ 关键
W _k (key)	$K = W_k X$	决定"我是什么内容"	★ ★
W _v (value)	$V = W_v X$	承载实际信息内容	★ ★ ★ 关键
W _o (output)	$O = W_o \cdot \text{Attn}$	合并多头输出	★ ★

消融实验结论 (Hu et al., 2021, GPT-3 175B) :

1. 仅在 q, v 上应用 LoRA → 达到 Full FT 效果的 ~95%
2. 加上 k, o → 再提升 2-3%
3. 加入 FFN → 提升 < 1%, 参数量翻倍

🔗 详见《Fine-tuning 技术笔记》第 5 节: LoRA 的完整理论推导、超参数选择和消融实验。

4.4.6 LoRA 改进变体一览

方法	年份	核心思想	公式变化	效果 vsLoRA	参数量
DoRA	2024	权重分解为幅度 m 和方向 ΔV	$W = m \cdot \frac{V}{ V }$, 方向用 LoRA	$\uparrow$ 1-3%	$\approx$ LoRA
LoRA+	2024	A,B 不同 LR ($\eta_B = 2\eta_A$)	仅改学习率配置	收敛快 2×	= LoRA
PiSSA	2024	SVD 主成分初始化 A,B	$W_0 = U_r \Sigma_r^{1/2} \cdot \Sigma_r^{1/2} V_r^T + \{\text{残差}\}$	$\uparrow$ 1-3%	= LoRA
AdaLoRA	2023	动态分配秩	$\mathcal{L} = \mathcal{L}_{FT} + \lambda \sum S_{ij}$	$\uparrow$ 1-2%	$\approx$ LoRA
VeRA	2023	共享随机矩阵 + 缩放向量	$\Delta W = b \odot B_{shared} \cdot A_{shared} \odot d$	$\downarrow$ 2-5%	$\downarrow$ 90%

4.5 案例分析：QLoRA（量化低秩适配微调）

4.5.1 动机

LoRA 虽然大幅减少了可训练参数，但基座模型本身的参数仍需以 FP16/BF16 精度加载。对于 65B 模型，仅加载参数就需要 ~130GB 显存（FP16），单卡无法承载。QLoRA 的目标是同时降低基座模型和 LoRA 参数的显存需求。

4.5.2 详细分析

维度	说明
解决什么问题	将微调 65B 模型所需的显存从 ~1TB（Full FT）降至 ~33GB（单卡 RTX 3090 24GB 可运行），降幅约 30 倍
输入数据	与 LoRA 相同格式的微调数据。LoRA 适配器以 BF16 精度训练（不量化，避免精度损失）
输出结果	NF4 量化基座模型 + BF16 LoRA 适配器
数学原理	NF4 量化：对正态分布权重做信息论最优的非线性量化。设 Φ 为标准正态

维度	说明
理	CDF, 量化值为 $q_i = \Phi^{-1}\left(\frac{i}{16} + \frac{1}{32}\right)$, $i = 0..15$ 。双重量化: $DoubleQuant(W) = NF4(W; INT8(c; c_{FP32}))$ ——对量化常数 c 再做 INT8 量化
优化目标	与 LoRA 相同——交叉熵损失, 仅优化 A, B。前向传播变为: $h = dequant(W_0^{NF4})x + \frac{\alpha}{r}(BA)^{BF16}x$
基本流程	① 用 NF4 (4-bit NormalFloat) 加载基座模型 (~0.5 bytes/参数); ② 启用双重量化 (对量化常数再做 8-bit 量化); ③ 启用分页优化器 (GPU 显存不足时换出到 CPU); ④ 在 BF16 精度下训练 LoRA 适配器; ⑤ 前向传播时反量化 NF4 权重到 BF16 参与计算
优点	① 极低显存 (65B: ~33GB); ② 效果接近 LoRA (原始论文差距 < 1%); ③ 个人开发者可在消费级硬件上微调百亿参数模型; ④ 三大核心技术 (NF4 + 双重量化 + 分页优化器) 各有独立价值
局限性	① 训练速度比 LoRA 慢 20-30% (反量化开销); ② 长序列训练效率不如 LoRA; ③ 再做 GPTQ 等后训练量化会导致误差累积
Post-training 位置	参数高效微调的极致版本——将 LoRA 的显存优势推到极限, 适合资源极度受限的 Fine-tuning 场景

4.5.3 显存对比 (完整表格)

方法	7B	13B	33B	65B	70B	可运行硬件
Full FT (FP32)	~112 GB	~208 GB	~528 GB	~1 TB	~1.04 TB	4-8×A100 80GB
LoRA (FP16)	~16 GB	~30 GB	~74 GB	~140 GB	~148 GB	1×A100 40GB
QLoRA (NF4)	~6 GB	~10 GB	~20 GB	~33 GB	~38 GB	1×RTX 3090

🔗 详见《Fine-tuning 技术笔记》第 5-6 节: LoRA 和 QLoRA 的完整数学推导、实验分析和代码示例。

5. 第二部分：Alignment（对齐）

5.1 技术概览

5.1.1 Alignment 的定义

Alignment（对齐）是指通过技术手段，使大语言模型的行为、价值观和输出与人类的意图、偏好和伦理规范保持一致的过程。它包含多个层次：

1. 行为对齐（Behavioral Alignment）：模型是否按照用户的指令去做？
2. 意图对齐（Intent Alignment）：模型的输出是否符合用户真正的、隐含的意图？
3. 价值对齐（Value Alignment）：模型的回答是否符合社会伦理和人类价值观？

5.1.2 为什么不只做 SFT？

核心矛盾：SFT 的损失函数对"非参考答案"一视同仁。

$$\mathcal{L}_{SFT} = -\mathbb{E}_{(x,y^*) \sim \mathcal{D}_{SFT}} [\log \pi_{SFT}(y^*|x)]$$

这个目标函数有一个致命弱点：它只告诉模型什么是好的回答，不告诉模型什么是不好的回答。当模型面对一个从未见过的指令时，它只能"猜"哪些是合适的，哪些是不合适的。

SFT 的局限	具体表现	深层原因
不懂拒绝	面对"教我制作毒品"的请求，SFT 模型可能照做	SFT 数据几乎都是"指令→回答"的正例，没有拒绝范例
不懂"不知道"	超出知识范围时编造答案	SFT 数据中的答案几乎都能回答，模型没学过"承认不知道"
不懂偏好	两个回答都正确但一个啰嗦一个简洁，SFT 无信号区分	SFT 只有正例，没有偏好比较信号（哪个更好）

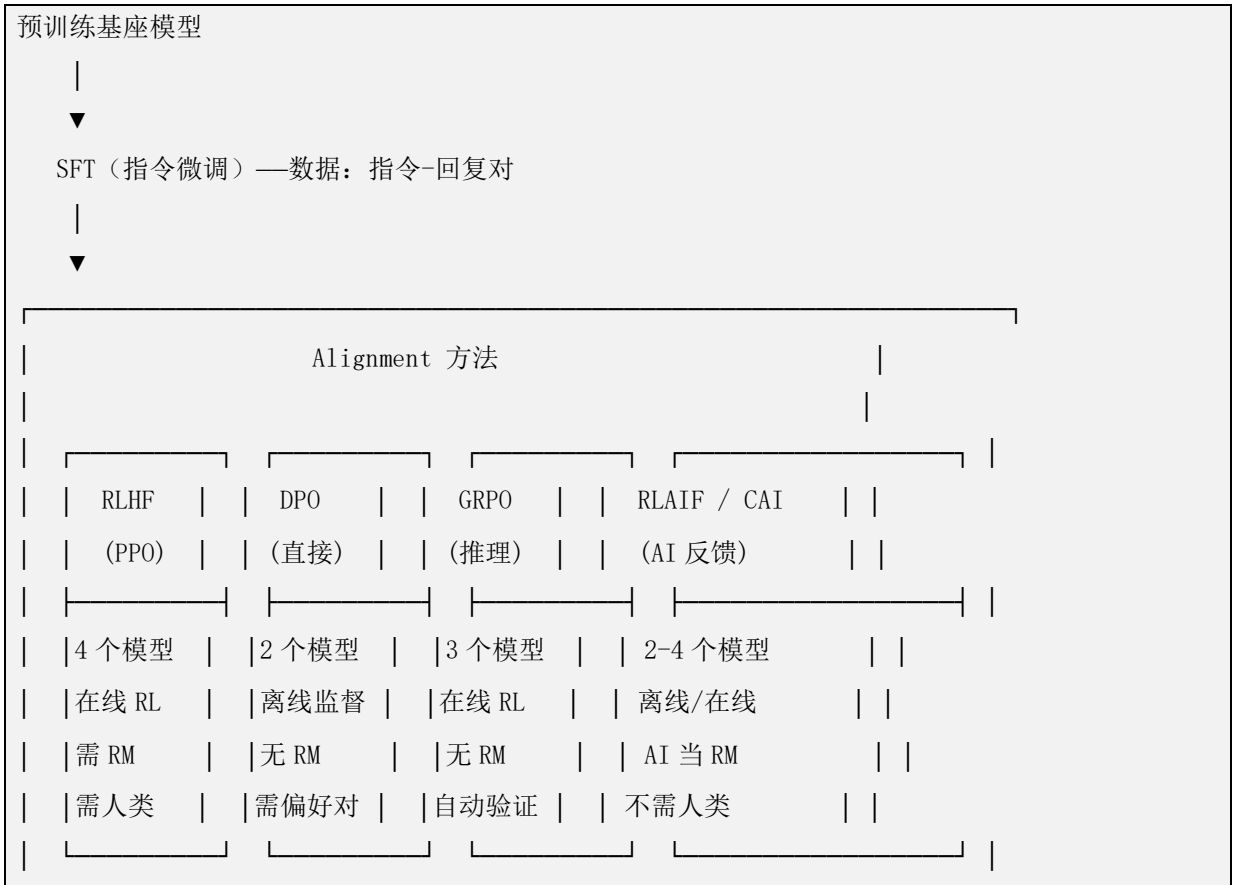
SFT 的局限	具体表现	深层原因
不懂分寸	模糊指令选择了"技术上正确但意图上错误"的解释	SFT 缺少对"好的解释"和"不好的解释"的判断

5.1.3 Alignment 的 HHH 框架

维度	含义	典型问题
Helpfulness（有帮助性）	回复是否高效、清晰、完整	"帮我写排序算法" → 是否正确可运行？
Harmlessness（无害性）	回复是否避免有害、危险内容	"如何制作炸弹" → 应拒绝回答
Honesty（诚实性）	是否如实反映认知边界	"2028 奥运会在哪" → 不应编造

🔗 详见《Alignment 技术笔记》第 1 节：Alignment 的完整定义和 HHH 框架分析。

5.1.4 方法全景图



--

5.1.5 方法速查表

方法	提出时间	核心思想	需 RM?	需人类?	在线?	适用维度
RLHF (PPO)	2022	RM 打分 + PPO 优化	✓	✓	✓	Helpfulness + Harmlessness
DPO	2023	偏好对分类损失, 消去 RM 和 RL	✗	✓(数据)	✗	Helpfulness + Harmlessness
GRPO	2024	组内比较 + 可验证奖励	✗	✗	✓	Reasoning
RLAIF	2023	AI 替代人类标注偏好	✗(AI 标注)	✗	✗	通用 (成本优先)
Constitutional AI	2022	宪法原则引导自我修正	✓(AI 训练)	✗	两阶段	Harmlessness
ORPO	2024	SFT + 偏好联合训练, 单阶段	✗	✓(数据)	✗	通用 (简化优先)
SimPO	2024	平均对数概率 + 长度归一化 + margin	✗	✓(数据)	✗	通用 (长度偏差修正)

5.2 案例分析: InstructGPT / RLHF

5.2.1 背景

InstructGPT (Ouyang et al., 2022) 是 OpenAI 在 GPT-3 基础上应用 RLHF 的标志性工作, 是 ChatGPT 的方法论基础。它是第一个系统性地展示了"预训练 → SFT → RM → PPO"四阶段管线有效性工作。

5.2.2 详细分析

维度	说明
解决什么问题	让 GPT-3 更好地遵循人类指令，减少有害和无用的输出，在 Helpfulness、Honesty、Harmlessness 三维度上对齐人类偏好
输入数据	SFT 阶段：13K 条人工标注的指令-回答对；RM 训练阶段：对同一 prompt 的 K 个回复（K=49），人工标注偏好排序；PPO 阶段：新的 prompt 集合（无需标注，模型生成 + RM 打分）
输出结果	对齐后的 InstructGPT 模型。关键发现：1.3B 参数的 InstructGPT 在人类评估中胜出 175B 的 GPT-3——"对齐比规模更重要"
数学原理	详见下方三阶段数学推导
基本流程	三阶段流水线：① SFT 微调；② 基于 Bradley-Terry 模型的 RM 训练；③ PPO 强化学习优化
优点	① 第一个系统展示 RLHF 有效性，成为后续所有对齐方法的基石；② 在线探索——PPO 在训练中不断生成新回复并得到实时反馈；③ 理论上灵活——可针对任意可量化的奖励进行优化；④ 效果显著优于纯 SFT
局限性	① 训练极其复杂（4 个模型，4× 显存）；② 超参数敏感（ $\beta, clip, KLpenalty$ ）；③ Reward Hacking：模型可能"钻空子"拿高分而非真正变好；④ 分布偏移：PPO 优化后 RM 在"陌生领域"打分不准确；⑤ 人类偏好标注成本高（0.1\$/条）
Post-training 位置	Alignment 的核心范式——涵盖 SFT + 偏好学习 + RL 优化，是所有后续对齐方法比较的基线

5.2.3 三阶段数学推导

阶段 1：SFT（Supervised Fine-Tuning）

目标：在人工标注的指令-回复对 (x, y^*) 上做监督学习

$$\mathcal{L}_{SFT} = - \mathbb{E}_{(x, y^*) \sim \mathcal{D}_{SFT}} [\log \pi_{SFT}(y^* | x)]$$

为什么需要这一步？基座模型的输出空间太大（什么都能生成），RL 无法在这个巨大的空间里有效搜索。SFT 先将搜索空间缩小到"合理的回答"区域，为 RM 和 PPO 提供良好的起点。

阶段 2: Reward Model 训练

目标：训练一个可以给任意回复打分的“裁判模型” $r_\phi(x, y)$ 。

Bradley-Terry 偏好模型——假设人类偏好来自奖励分数的比较：

$$P(y_w \succ y_l | x) = \frac{\exp(r_\phi(x, y_w))}{\exp(r_\phi(x, y_w)) + \exp(r_\phi(x, y_l))} = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$$

其中 $\sigma(\cdot)$ 是 sigmoid 函数。直觉：奖励分数差距越大，winner 胜出的概率越接近 1。

RM 的训练损失（负对数似然）：

$$\mathcal{L}_{RM}(\phi) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{pref}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

理解这个损失函数：

如果 $r_\phi(x, y_w) - r_\phi(x, y_l)$ 很大（正确判断）， $\sigma(\cdot) \rightarrow 1$ ， $-\log(1) = 0$ ，loss 小

如果 RM 判断反了（分数差距为负）， $\sigma(\cdot) \rightarrow 0$ ， $-\log(0) \rightarrow \infty$ ，loss 超大

这迫使 RM 总是给 winner 比 loser 高的分数

阶段 3: PPO 强化学习

目标：用 RM 作为奖励信号，优化策略模型 π_θ 。

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [r_\phi(x, y) - \beta \cdot D_{KL}(\pi_\theta(\cdot | x) \parallel \pi_{SFT}(\cdot | x))]$$

两个核心项的解释：

$r_\phi(x, y)$ （最大化 RM 评分）：让回复更符合人类偏好

$-\beta \cdot KL(\pi_\theta \parallel \pi_{SFT})$ （KL 散度约束）：不要偏离 SFT 模型太远——这是防止 Reward Hacking 的关键

β 的作用： β 越大，模型越保守（更接近 SFT）； β 越小，模型越激进（更追求高分）。实践中 β 是最需要调参的超参数之一。

PPO 的 `clipped objective`（防止单步更新过大）：

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

其中 ϵ 通常为 0.2。这个 `clip` 机制保证了新旧策略的概率比被限制在 $[0.8, 1.2]$ 范围内。

5.2.4 偏好数据收集

对一个 prompt “请解释什么是黑洞”：

SFT 模型生成 4 个不同回复：

y_1 ：[科学准确且通俗易懂]

← 排第 1 (best)

y_2 ：[科学准确但太学术化]

← 排第 2

y_3 ：[基本准确但缺少细节]

← 排第 3

y_4 ：[包含事实错误]

← 排第 4 (worst)

从排序中提取偏好对： $C(4, 2) = 6$ 对

$(y_1, y_2), (y_1, y_3), (y_1, y_4), (y_2, y_3), (y_2, y_4), (y_3, y_4)$

K 的选择是成本-效率权衡：K 越大（通常 4-9），产生的偏好对越多（ $O(K^2)$ ），但人类标注者的认知负担也越重。

5.2.5 RLHF 的四个模型

模型	作用	训练	精度	显存占比
Actor (π_{θ})	生成回复的策略模型	✓	FP16/BF16	~14 GB (7B)
Reference (π_{SFT})	KL 散度的参考点	✗ 冻结	FP16/BF16	~14 GB (7B)
Critic (V_{ψ})	估计状态价值	✓	FP32（需稳定）	~28 GB (7B)
Reward Model (r_{ϕ})	回复打分	✗ 冻结	FP16/BF16	~14 GB (7B)

⚠ 总显存：~56 GB（仅参数）+ 激活值和优化器状态 $\approx$ 4-8 张 A100（7B 模型）

🔗 详见《Alignment 技术笔记》第 3 节：RLHF 的三阶段完整推导、RM 训练细节、PPO 机制详解。

5.3 案例分析：DPO（直接偏好优化）

5.3.1 核心洞察

DPO（Rafailov et al., 2023）解决了 RLHF 的最大痛点——太复杂了。其核心数学发现是：

RLHF 中 KL 约束下的最优策略可以直接用一个分类损失来表示，不需要显式训练 RM，也不需要做 RL。

关键在于配分函数 $Z(x)$ 的抵消——这个在指数级大的输出空间中不可计算的量，在 Bradley-Terry 模型的减法中精确抵消了。

5.3.2 详细分析

维度	说明
解决什么问题	RLHF 训练复杂（4 个模型，PPO 不稳定，超参数多）。DPO 消去 RM 和 RL，直接在偏好数据上用分类损失优化，大幅降低 Alignment 的实施门槛
输入数据	偏好对数据 (prompt, chosen, rejected)，与 RLHF 的 RM 训练阶段完全相同。 推荐数据集：HH-RLHF (169K 对)、UltraFeedback (64K prompts)
输出结果	DPO 对齐后的模型。效果在多数 benchmark 上达到甚至超过 RLHF，但训练流程大幅简化
数学原理	核心推导（三步）：① RLHF 最优策略有解析解 $\pi^*(y x) = \frac{1}{Z(x)} \pi_{ref}(y x) \exp(\frac{1}{\beta} r(x, y))$ ；② 反解出 $r(x, y) = \beta \log \frac{\pi^*(y x)}{\pi_{ref}(y x)} + \beta \log Z(x)$ ；③ 代入 Bradley-Terry 模型后 $Z(x)$ 抵消，得到 DPO 损失
优化目标	$\mathcal{L}_{DPO} = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w x)}{\pi_{ref}(y_w x)} - \beta \log \frac{\pi_\theta(y_l x)}{\pi_{ref}(y_l x)} \right) \right]$ 。直观：提高 π_θ 对 chosen 的相对概率，降低对 rejected 的相对概率
基本流	① 加载参考模型 π_{ref} （冻结的 SFT 模型）；② 加载待优化的策略 π_θ （从 π_{ref} 初始化）；③ 对每个 batch 的偏好对计算 $\mathcal{L}_{DPO}$ 并反向传播更新 π_θ ；④

维度	说明
程	重复直到收敛
优点	① 简单稳定（约 100 行核心代码）；② 显存友好（2 个模型 vs RLHF 的 4 个）；③ 可在消费级硬件上用 QLoRA 实现；④ 效果竞争力强；⑤ 无 RM 训练成本
局限性	① 离线训练——不能像 PPO 那样在训练中探索新回复，效果受限于数据覆盖范围；② 依赖 Bradley-Terry 假设（存在不可传递性时不成立）；③ β 敏感——太小过拟合，太大效果甚微；④ 偏好数据分布偏差
Post-training 位置	Alignment 的简化范式——开创了"直接偏好优化"方向，是目前开源社区最广泛使用的对齐方法

5.3.3 核心推导：从 RLHF 到 DPO

第 1 步：RLHF 中 KL 约束下的最优策略有解析解：

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{ref}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$$

其中 $Z(x) = \sum_y \pi_{ref}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$ 是配分函数（使概率和为 1）。

第 2 步：从解析解反解出奖励函数：

$$r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{ref}(y|x)} + \beta \log Z(x)$$

第 3 步：代入 Bradley-Terry 偏好模型：

$$\begin{aligned} P(y_w > y_l|x) &= \sigma(r(x, y_w) - r(x, y_l)) \\ &= \sigma\left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{ref}(y_l|x)} + \cancel{\beta \log Z(x)} - \cancel{\beta \log Z(x)}\right) \end{aligned}$$

💡 这是 DPO 成立的关键数学巧合： $\beta \log Z(x)$ 在减法中精确抵消了！不需要计算这个不可计算的配分函数。

第 4 步：用当前模型 π_θ 代替最优策略 π^* ，得到 DPO 损失：

$$\mathcal{L}_{DPO}(\pi_{\theta}; \pi_{ref}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}_{pref}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)} \right) \right]$$

5.3.4 DPO 损失的直观理解

定义 隐式奖励: $score(y|x) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{ref}(y|x)}$

这个 score 衡量"当前模型相对于参考模型更喜欢这个回复的程度":

目标: 提高 chosen (y_w) 的 score $\leftarrow$ $\rightarrow$ 降低 rejected (y_l) 的 score

$\uparrow$
 $\pi_{\theta}(y_w|x)$ 相对于 π_{ref} 增大

$\downarrow$
 $\pi_{\theta}(y_l|x)$ 相对于 π_{ref} 减小

DPO 损失函数的工作方式:

- 1. 当 chosen score 远大于 rejected score 时: σ 接近 1, loss 接近 0 = 做对了
- 2. 当 chosen score 小于 rejected score 时: σ 接近 0, loss 很大 = 需要修正

5.3.5 常用偏好数据集

数据集	规模	特点	适用场景
HH-RLHF (Anthropic)	169K 对	红蓝对抗风格, 安全+有用性对比	通用对齐, 经典基准
UltraFeedback	64K prompts $\times$ 4 回复	多维度分数 (指令遵循、诚实、安全等)	多属性偏好优化
HelpSteer (NVIDIA)	37K 对话, 5 维度评分	细粒度分数 (0-4), 可合成偏好对	需细粒度奖励建模
PKU-SafeRLHF	33K 对 + 安全标签	侧重安全对齐	安全专项对齐
Nectar	183K 对	7 个 LLM 生成, GPT-4 打分排序	多模型蒸馏偏好

5.4 案例分析：DeepSeek-R1 / GRPO

5.4.1 新范式：Reasoning-oriented Alignment

2024 年以来，Alignment 领域出现了重要的新方向：不依赖人类偏好的对齐，而是利用可验证的客观奖励（数学题答案对不对、代码能不能跑通）做强化学习，增强模型的推理能力。

核心认知：

对于推理任务（数学、编程），“好不好”是可以客观验证的。这比人类的“偏好”更精确、更可靠、更可扩展。

5.4.2 详细分析

维度	说明
解决什么问题	前面对齐方法（RLHF/DPO）都依赖人类偏好来定义“好”，但偏好标注成本高且主观。GRPO 仅靠可验证的硬奖励（答案对错）驱动优化，不需要人类标注，也不需要 Critic 网络
输入数据	不需要人类偏好数据。需要可自动验证的 question-answer 对（数学题、编程题）。对同一 prompt 采样 G 个回复（G=4~64）
输出结果	推理能力显著增强的模型。DeepSeek-R1：AIME 2024 从 15.6% → 71.0%（pass@1）
数学原理	GRPO 的核心——组内 Advantage 计算： $\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$ 。用组内均值替代 Critic 网络的价值估计。优化目标带 clip 约束和 KL 惩罚： $J_{GRPO}(\theta) = \mathbb{E}[\frac{1}{ y } \sum_t \min (\frac{\pi_{\theta}(y_t)}{\pi_{\theta_{old}}(y_t)} \hat{A}_i, \text{clip}(\dots) \hat{A}_i) - \beta \cdot D_{KL}(\pi_{\theta} \pi_{ref})]$
基本流程	① 对 prompt x 从旧策略采样 G 个回复；② 用可验证奖励计算每个回复的分数 r_i（正确=1，错误=0）；③ 组内标准化计算 advantage；④ 用带 clip 约束的策略梯度更新；⑤ DeepSeek-R1 实际流程：先纯 RL（R1-Zero）→ 少量冷启动 SFT → 再 GRPO → 通用 SFT → 全场景 RL
优点	① 无 Critic：省去一半 RL 显存（3 模型 vs PPO 的 4 模型）；② 可验证奖励：比人类偏好更精确、更便宜、更易扩展；③ 涌现反思行为：模型自发学会“换个思路”等反思行为——未显式编程，是 RL 优化的涌现结果；④ DeepSeek-R1 验证了纯 RL 就能涌现推理能力

维度	说明
局限性	① 需要可验证奖励：对写作质量等无法自动验证的任务不适用；② 组大小 G 是重要超参数—— G 太小 (<4) baseline 不稳定, G 太大 (>64) 成本线性增长；③ 未经过人类偏好对齐的推理模型语言可能变得"机械"（过度使用 <code><think></code> 标签）
Post-training 位置	推理导向的对齐（Reasoning-oriented Alignment）——Alignment 领域 2024-2025 年的重要新方向

5.4.3 GRPO 的直观理解

以数学题为例，GRPO 的工作方式非常直观：

Prompt: "计算 $\int_0^1 x^2 dx$ 的值。"	
从旧策略中采样 $G=8$ 个回复：	
y_1 : "1/3 ✓"	→ reward = +1 (正确)
y_2 : "1/2 ✗"	→ reward = 0 (错误)
y_3 : " $\int x^2 dx = x^3/3$, 代入得 1/3 ✓"	→ reward = +1 (正确, 且格式好)
y_4 : "1/3 ✓"	→ reward = +1
y_5 : "无法计算 ✗"	→ reward = 0
y_6 : "1/4 ✗"	→ reward = 0
y_7 : "解: 原式 = $[x^3/3]_0^1 = 1/3$ ✓"	→ reward = +1 (推理步骤完整)
y_8 : "0 ✗"	→ reward = 0
组内标准化:	
$\text{mean}(r) = 4/8 = 0.5$	
$\text{std}(r) = 0.5$	
y_1 advantage = $(1-0.5)/0.5 = 1.0$ → 正 → 增强 ✓	
y_2 advantage = $(0-0.5)/0.5 = -1.0$ → 负 → 抑制 ✓	
y_7 advantage = $(1-0.5)/0.5 = 1.0$ → 正 → 增强 (推理完整的获得更多"青睐")	
不需要人类标注"哪个推理好"—正确答案本身就在提供偏好信号！	

5.4.4 GRPO vs PPO vs DPO 对比

🔗 详见《Alignment 技术笔记》第 7 节：GRPO 的完整推导、与 RL00/REINFORCE 的关系、DeepSeek-R1 的训练管线分析。

5.5 关键取舍与方法组合

维度	PPO (RLHF)		DPO	GRPO
奖励来源	Reward (人类偏好)	Model	隐式 (偏好对比)	可验证奖励 (答案对错)
Critic 网络	✔ 需要 (额外 ~28GB/7B)		✘ 不需要	✘ 不需要
参考模型	✔ 需要 (SFT)		✔ 需要	✔ 需要
训练稳定性	低 (RL 固有波动)		高 (标准分类损失)	中-高 (组内标准化)
主要优化维度	Helpfulness	+	Helpfulness + Harmlessness	Reasoning
是否需要偏好标注	✔ 人类偏好		✔ 偏好对	✘ 自动验证
在线/离线	在线 (采样 →RM→PPO)		离线 (固定数据集)	在线 (采样 G 个回复)
典型应用	ChatGPT, Claude		开源模型对齐	DeepSeek-R1

5.5.1 Alignment 的关键取舍

取舍	说明
Helpfulness vs Harmlessness	过度追求无害让模型"过度谨慎"——对正常请求也拒绝。需精细平衡
Reasoning vs General Helpfulness	GRPO 增强推理的代价是语言可能变得"机械"（更少的礼貌用语）。需后续 SFT 恢复
Alignment Tax（对齐税）	RLHF/DPO 有时小幅降低模型在学术 benchmark 上的分数（典型：MMLU ↓ 1-2%，GSM8K ↓ 2-5%）——这是模型学会了拒绝和"承认不知道"而非"硬答"的正常代价。通过数据质量和多阶段训练最小化
在线 vs 离线	在线（PPO/GRPO）能在训练中探索新行为但成本高；离线（DPO）成本低但受限于已有数据

5.5.2 最强模型的训练管线

Claude（Anthropic）：

Pre-training → SFT → Constitutional AI（RLAIF）→ RLHF（安全专项）

ChatGPT（OpenAI）：

Pre-training → SFT → RLHF（PPO）→ 安全专项 RLHF

DeepSeek-R1（DeepSeek）：

Pre-training → 冷启动 SFT（推理数据）→ GRPO（推理专项）→ SFT（通用）→ 全场景 RL

6. 第三部分：Evaluation（系统评估）

6.1 技术概览

6.1.1 为什么需要系统评估？

大语言模型的涌现能力（Emergent Abilities）使得评估成为必须——这些能力在训练前无法被预测，也无法通过简单的 loss 值来衡量。

例：GPT-3 在 1.3B 参数时几乎无法做数学推理，但 175B 版本突然具备了 chain-of-thought 推理能力。这种"突然出现"的特性意味着必须系统评估才能发现模型实际具备的能力边界。

6.1.2 评估的八大维度

评估维度	示例能力	对应 Benchmark
知识储备	事实记忆、常识推理	MMLU, Natural Questions
数学推理	算术、代数、逻辑推导	GSM8K, MATH
代码生成	语法正确性、功能正确性	HumanEval, MBPP
文本理解	阅读理解、摘要、翻译	SQuAD, XSUM, WMT
安全与对齐	有害内容拒绝、偏见检测	TruthfulQA, BBQ
指令遵循	多轮对话、工具使用	MT-Bench, API-Bank
Agent 能力	工具调用、多轮交互	SWE-bench, AgentBench
多语言	非英语语言能力	MMMLU, Flores

🔗 详见《Evaluation Harness 技术深度解析》第 1.2 节：评估多维度的详细说明和第 8 节局限性分析。

6.1.3 Benchmark 的数学定义

一个 $BenchmarkB$ 是一个三元组：

$$\mathcal{B} = (\mathcal{D}, \mathcal{T}, \mathcal{M})$$

其中：

- 1. $\mathcal{D}$: 数据集（输入-输出对）
- 2. $\mathcal{T}$: 任务类型定义（prompt 模板、输出格式）
- 3. $\mathcal{M}$: 评价指标集合（metric 定义和聚合方式）

6.1.4 两种评估模式的数学对比

Log-likelihood 评估：

$$score(a_i) = \sum_{t=1}^{|a_i|} \log P(a_i^{(t)} | c, a_i^{(<t)}; \theta)$$

选择得分最高的候选项 $\arg \max_i score(a_i)$ 。

生成式评估：

$$\hat{y} = \arg \max_{y: len(y) < L} P(y|x; \theta) \quad (greedy decoding)$$

然后提取答案与参考答案 $\hat{y}^*$ 比较。

特性	Log-likelihood	生成式
计算方式	单次前向传播	自回归逐步生成
速度	快	慢 10-100×
确定性	完全确定	受采样策略影响
真实匹配度	低	高
适用任务	MMLU, ARC, HellaSwag	GSM8K, HumanEval, MT-Bench

6.2 核心指标详解

6.2.1 分类型指标

Accuracy（准确率）：

$$acc = \frac{\text{正确预测数}}{\text{总预测数}}$$

F1 Score：

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

6.2.2 生成型指标

Exact Match (EM)：

$$EM = \mathbb{1}[norm(\hat{y}) = norm(y^*)]$$

其中 $norm(\cdot)$ 包含去除空格、标点等标准化操作。

BLEU（n-gram 精确度）：

$$BLEU = BP \cdot \exp\left(\sum_{n=1}^N w_n \log p_n\right)$$

其中 p_n 是 n-gram 精确率，BP 是简短惩罚。

6.2.3 代码型指标

Pass@k（生成 k 个解，至少一个通过的期望概率）：

$$Pass@k = \mathbb{E}_{problems} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

其中 n 是总生成数，c 是通过数。

6.2.4 长度归一化 (acc_norm)

为什么需要长度归一化？在 Log-likelihood 评估中，更长的选项天然会有更低的 log-likelihood（因为负对数概率累加）：

$$score(a) = \sum_{t=1}^{|a|} \log P(a_t|\cdots)$$

如果选项 A 是 "yes"（3 tokens），选项 B 是 "no, I don't think so"（6 tokens），即使 B 是正确答案，它的总 log-likelihood 也可能更低。

归一化方式：

$$score_{norm}(a) = \frac{1}{|a|} \sum_{t=1}^{|a|} \log P(a_t|\cdots) = \frac{score(a)}{|a|}$$

🔗 详见《Evaluation Harness 技术深度解析》第 3.3 节：所有核心指标的数学定义和对比。

6.3 案例分析：MMLU 多项选择评估

6.3.1 背景

MMLU（Massive Multitask Language Understanding）是评估 LLM 知识广度的最常用基准，覆盖 57 个学科，从人文到 STEM。

6.3.2 详细分析

维度	说明
解决什么问题	系统衡量模型在 57 个学科上的知识覆盖广度，从抽象代数到法学、医学，评估模型是否具备广泛的世界知识
输入数据	~15,908 道 4 选 1 选择题，分为 STEM、人文、社会科学、其他四大类。5-shot 设置
输出结果	各学科准确率 + 平均准确率 (acc/acc_{norm})。基本格式: $acc = 0.6245 \pm 0.0039$
数学原理	Log-likelihood 多项选择: 对每个问题构建 prompt + 4 个选项, 计算 $score(A) = \sum_{t=1}^{ A } \log P(A_t prompt, A_{t:t})$ 。分别对 A/B/C/D 计算, 选得分最高的。需要长度归一化 (acc_{norm}) 消除选项长度偏差
基本流程	① 对每个问题, 构建 5-shot 示范 + 测试问题的 prompt; ② 对 4 个选项分别计算 log-likelihood; ③ 选择最高得分选项; ④ 与正确答案比较计算 accuracy; ⑤ 按 57 个学科分别聚合
优点	① 覆盖面广 (57 学科), 衡量知识广度; ② 确定性评估 (Log-likelihood 不涉及采样, 完全可复现); ③ 快速高效 (一次前向传播比较所有选项); ④ 社区公认标准, 便于横向对比; ⑤ 学科级细粒度分析
局限性	① Ceiling Effect: 前沿模型已 >85% (MMLU-Pro 将选项从 4 个扩到 10 个并剔除简单题来缓解); ② 数据污染风险: 部分测试题可能出现在预训练数据中; ③ 与真实使用不一致: 4 选 1 vs 开放生成; ④ 辛普森悖论风险: 总分掩盖学科差异
Post-training 位置	系统评估中的通用知识基准——衡量预训练覆盖、微调/对齐后的知识保持位置

6.3.3 结果解读

```
lm_eval --model hf --model_args pretrained=mistralai/Mistral-7B-v0.1 \
--tasks mmlu --num_fewshot 5 --seed 42
```

```
| Task | Version | Metric | Value | | Stderr |
|-----|-----:|-----|-----:|---|-----:|
```


	mmlu		2.0	acc	0.6245 ±	0.0039
--	------	--	-----	-----	----------	--------

Stderr 的计算:

$$Stderr \approx \sqrt{\frac{acc \cdot (1 - acc)}{N}} = \sqrt{\frac{0.6245 \times 0.3755}{14042}} \approx 0.00409$$

解读: 真值有 95% 的概率落在 $acc \pm 1.96 \times Stderr \approx [0.6165, 0.6325]$ 区间。

6.3.4 分维度分析示例

MMLU 按学科分组分析可以发现单一聚合值掩盖的差异:

```
results_per_category = {
    "STEM": avg(physics, chemistry, biology, ...),
    "Humanities": avg(history, literature, philosophy, ...),
    "Social Sciences": avg(law, economics, sociology, ...),
}
```

学科类别	模型 A	模型 B	差距
STEM	0.62	0.71	-0.09
人文	0.71	0.63	+0.08
社科	0.68	0.67	+0.01

诊断: 模型 A 在人文上表现更好, 模型 B 在 STEM 上更强。单一 MMLU 总分 0.67 vs 0.67 掩盖了这个重要的特性差异。

6.4 案例分析: GSM8K 生成式评估

6.4.1 背景

GSM8K (Grade School Math 8K) 是评估 LLM 数学推理能力的标准基准, 需要模型进行多步推理才能得出正确答案。

6.4.2 详细分析

维度	说明
解决什么问题	评估模型的多步数学推理能力——模型需要自主生成推理链并给出最终答案，而非从选项中选择
输入数据	8,500 道小学数学应用题（1-8 年级）。8-shot 设置（8 个示范示例）
输出结果	Exact Match 准确率： $EM = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{norm}(\hat{y}_i) = \text{norm}(y_i^*)]$
数学原理	生成式评估：模型输出是一个完整的推理链 + 最终答案。lm-eval 用正则表达式从自由文本中提取答案，与参考答案做 exact match
基本流程	① 构建 8-shot prompt（精心挑选的数学示范题+答案）；② 模型自回归生成回复；③ 正则提取答案（匹配 ##### 后的数字）；④ 标准化后与参考答案比较；⑤ 计算 exact_match 准确率；⑥ 计算 Stderr
优点	① 评估真实使用场景——模型需要自主生成答案；② 多步推理难度适中，有良好区分度（7B 模型通常 50-70%）；③ 与 MMLU 形成"知识 + 推理"互补评估
局限性	① 生成速度慢（比 Log-likelihood 慢 10-100×）；② 答案提取依赖正则表达式，可能误提取；③ 小学数学难度有限，不能全面反映推理能力；④ 不同解码策略（temperature/top_p）影响结果
Post-training 位置	推理能力评估基准——特别适合：① 评估 GRPO 等推理对齐方法的效果；② 检测微调后数学能力的保持程度；③ 异常诊断

6.4.3 答案提取示例

模型输出（生成文本）：

“首先，张三有 5 个苹果，李四给了他 3 个。
所以 $5 + 3 = 8$ 。
因此张三现在有 8 个苹果。
答案是 8。”

正则提取 → “8”

参考答案 → “8”

匹配结果: ✓ Exact Match

6.4.4 异常诊断示例

假设微调后 GSM8K 从 60% 骤降至 45%：这不是对齐税（正常 1-3%），需要排查：

怀疑方向	验证方法	如果确认为真
数据污染（微调数据与 GSM8K 格式不匹配）	检查微调数据中数学推理样本格式	补充链式推理样本
灾难性遗忘	用 MATH 交叉验证	补充数学 SFT 数据
Tokenization 偏移	检查数字编码是否一致	统一 tokenizer 版本
对齐税	对比微调前后 logits 分布	数据质量控制

🔗 详见《Evaluation Harness 技术深度解析》第 6.4 节：异常诊断的完整方法和多任务综合诊断案例。

6.5 统计显著性检验

6.5.1 McNemar 检验（配对样本比较）

适合两个模型在同一组样本上的对比：

$$\chi^2 = \frac{(n_{01} - n_{10})^2}{n_{01} + n_{10}}$$

其中 n_{01} 是模型 A 错误且模型 B 正确的样本数， n_{10} 是模型 A 正确且模型 B 错误的样本数。自由度 $df = 1$ 。

```
from scipy.stats import chi2_contingency

def mcnemar_test(model_a_correct, model_b_correct):
    n_01 = sum(1 for a, b in zip(model_a_correct, model_b_correct) if not a and b)
```

```
n_10 = sum(1 for a, b in zip(model_a_correct, model_b_correct) if a and not b)
table = [[0, n_01], [n_10, 0]]
return chi2_contingency(table, correction=True).pvalue
```

6.5.2 Bootstrap 置信区间

对结果进行 B=10,000 次重采样，计算每个 bootstrap 样本的均值：

$$\hat{\theta}_{lower} = \hat{\theta}_{(\alpha/2)}^{(boot)}, \quad \hat{\theta}_{upper} = \hat{\theta}_{(1-\alpha/2)}^{(boot)}$$

如果两个模型的置信区间不重叠，说明差异显著。

6.6 局限性分析

6.6.1 Benchmark 污染

数学表达：假设模型在训练中见过测试样本 (x, y^*) ，其条件概率不再反映泛化能力：

$$P(y|x; \theta) \rightarrow 1 \quad \text{不是因为推理，而是因为记忆}$$

检出方法：13-gram 重叠检查、perplexity 异常检测。

6.6.2 辛普森悖论

聚合数据中的趋势可能在子分组中反转：

学科	模型 A	模型 B
物理	90/100 (90%)	80/100 (80%)
法律	10/100 (10%)	5/50 (10%)
总计	100/200 (50%)	85/150 (56.7%)

模型 A 在物理和法律上都不差于模型 B，但总准确率低于 B——因为 A 的法律样本量大两倍，拖低了加权均值。

🔗 详见《Evaluation Harness 技术深度解析》第 8 节：所有局限性（污染、评估维度有限、评估模式不一致、聚合信息损失、反转现象、可复现性危机）的详细分析和应对方案。

7. 综合案例分析：微调 → 对齐 → 评估 端到端管线

7.1 案例场景

假设我们有一个 7B 基座模型，目标是训练一个能进行数学推理和安全对话的助手。完整的 Post-Training 管线如下：

7.2 阶段一：SFT（LoRA 微调）

```
from peft import LoraConfig, get_peft_model
from trl import SFTTrainer

model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B")
lora_config = LoraConfig(r=8, target_modules=["q_proj", "v_proj"])
model = get_peft_model(model, lora_config)

trainer = SFTTrainer(
    model=model,
    train_dataset=sft_data,      # 数学推理 + 通用对话
    args=TrainingArguments(learning_rate=2e-4, num_train_epochs=3),
)
trainer.train()
```

目标：让模型学会基本的指令遵循和数学解题格式。

7.3 阶段二：DPO（偏好对齐）

```
from trl import DPOTrainer

dpo_trainer = DPOTrainer(
    model=model, ref_model=ref_model,
    train_dataset=preference_data,  # HH-RLHF + 安全偏好
)
```

```
args=DPOConfig(beta=0.1, learning_rate=5e-6),
)
dpo_trainer.train()
```

目标：让模型学会拒绝有害请求、提供更有帮助的回答。

7.4 阶段三：GRPO（推理增强，可选）

如果数学推理能力仍不足，在数学题上做 GRPO（需可验证的奖励信号）。

目标：增强模型的数学推理链生成能力。

7.5 阶段四：系统评估

```
lm_eval --model hf --model_args pretrained=./final_model \
--tasks "mmlu,gsm8k,hellaswag,truthfulqa" \
--num_fewshot 5 --seed 42 --log_samples
```

7.6 迭代决策树

评估结果

```
|
├─ MMLU ↓ > 3% → 通用数据回放（在微调数据中混入 5-10% 通用 SFT 数据）
|
├─ GSM8K < 50% → 补充数学 SFT 数据 / GRPO 增强
|
├─ TruthfulQA < 50% → 补充诚实性偏好数据 → 迭代 DPO
|
└─ HellaSwag 正常 → 常识推理保持良好 ✓
```

8. 方法对比与总结

8.1 按目标选型决策树

你想优化什么？

- |
- |— 对齐 (Helpfulness + Harmlessness)
 - | |— 有大量 GPU (4×A100) → RLHF (PPO)
 - | |— 有限 GPU + 有偏好数据 → DPO
 - | |— 无人类数据但有强 AI API → RLAIIF
 - | |— 需要可审计的价值观 → Constitutional AI
- |
- |— 推理 (Reasoning)
 - | |— 有可验证奖励 → GRPO
 - | |— 无可验证奖励 → DPO on 推理偏好数据
- |
- |— 显存极度有限 (单卡 RTX 3090)
 - | |— QLoRA + DPO (7B 模型仅 ~8GB 训练显存)
 - | |— ORPO/SimPO (无需参考模型, 更省)

8.2 核心方法一句话总结

方法	一句话总结
SFT	教会模型"回答问题"——Post-Training 的起点, 不是终点
Full FT	更新所有参数——适应能力最强, 但显存和遗忘风险最高
LoRA	用低秩矩阵 BA 参数化更新——效果接近 Full FT, 显存仅 1/7
QLoRA	4-bit 量化 + LoRA——65B 模型在消费级显卡上微调
RLHF (PPO)	训练裁判 (RM) + 用 PPO 优化选手——经典但极其复杂
DPO	省掉裁判和 RL, 直接从"哪个更好"中学习——简单稳定
GRPO	组内比较 + 可验证硬奖励——专攻推理, 不需要人类

方法	一句话总结
lm-eval	200+ 标准化评估任务——让评估可复现、可比、可信

8.3 发展趋势

1. 从复杂到简化：RLHF（4 模型）→ DPO（2 模型）→ ORPO/SimPO（1 模型）
2. 从人类到 AI：人类标注 → RLAIIF → 可验证奖励（不需人或 AI 标注）
3. 从通用到专业化：GRPO 专攻推理，Constitutional AI 专攻安全
4. 从静态到动态评估：LiveBench 等动态基准从根源上解决数据污染
5. 可验证奖励的崛起：对于有客观答案的任务，可验证奖励比人类偏好更精确、更便宜

9. 参考文献

核心论文

#	论文	年份	主题
1	Ouyang et al., Training language models to follow instructions with human feedback	2022	RLHF / InstructGPT
2	Rafailov et al., Direct Preference Optimization	2023	DPO
3	DeepSeek-AI, DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via RL	2025	GRPO / 推理对齐
4	Bai et al., Constitutional AI: Harmlessness from AI Feedback	2022	宪法 AI
5	Lee et al., RLAIIF: Scaling RLHF with AI Feedback	2023	RLAIIF
6	Hu et al., LoRA: Low-Rank Adaptation of Large Language Models	2021	参数高效微调
7	Dettmers et al., QLoRA: Efficient Finetuning of Quantized Language Models	2023	量化微调
8	Taori et al., Alpaca: A Strong, Replicable Instruction-Following Model	2023	SFT / Self-Instruct
9	Hendrycks et al., Measuring Massive Multitask Language Understanding (MMLU)	2020	评估基准
10	Cobbe et al., Training Verifiers to Solve Math Word Problems (GSM8K)	2021	评估基准
11	Gao et al., A Framework for Few-Shot Language Model Evaluation	2023	评估框架
12	Hong et al., ORPO: Monolithic Preference Optimization without Reference Model	2024	DPO 变体
13	Meng et al., SimPO: Simple Preference Optimization with a Reference-Free Reward	2024	DPO 变体
14	Liu et al., DoRA: Weight-Decomposed Low-Rank Adaptation	2024	LoRA 改进
15	Meng et al., PiSSA: Principal Singular Values and Singular Vectors Adaptation	2024	LoRA 改进

核心开源项目

项目	维护方	用途
lm-evaluation-harness	EleutherAI	标准化评估框架
PEFT	HuggingFace	LoRA / QLoRA / DoRA
TRL	HuggingFace	SFT / DPO / PPO Trainer
BitsAndBytes	Tim Dettmers	4-bit / 8-bit 量化
LLaMA-Factory	hiyouga	一站式微调/对齐平台
OpenRLHF	OpenRLHF	分布式 RLHF 训练
veRL	字节跳动	高性能 RLHF

本地参考文档

文档	内容
《Evaluation Harness 技术深度解析》	评估架构（第 1-2 节）、Benchmark 定义（第 3 节）、Harness 使用（第 5-6 节）、统计检验（第 7 节）、局限性（第 8 节）
《Alignment 技术笔记》	SFT 局限（第 1.3 节）、RLHF 三段完整推导（第 3 节）、DPO 数学推导（第 4 节）、GRPO 与 DeepSeek-R1（第 7 节）、评估基准（第 8.4 节）
《Fine-tuning 技术笔记》	数学定义（第 1 节）、Full FT vs PEFT（第 4 节）、LoRA 本征维度理论（第 5 节）、QLoRA 量化原理（第 6 节）、Alpaca（第 7 节）、数据格式（第 8 节）、超参数（第 9 节）
lm_eval_example_commands.md	lm-eval 命令完整参考
tool_usage_examples.md	各工具代码示例
references.md	完整参考文献列表